

Working with different data types

Data types, what they are, why they are important (R predominantly)

ASI WA-branch workshop 10th July 2026

Dr Aaron B Beasley



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

Key concepts: types, functions, loops

- Dtypes are types of data (R calls these classes)
- R by default stores all numeric data as double precision (64 bit)
 - 1, 1.1, 0.001 – all in 64 bit
 - Can be a bit wasteful but not really an issue unless extremely memory or time constrained
- Floating point math!
 - Try this in R $\rightarrow (0.1 + 0.2) == 0.3$
 - What happened?
- Why are dtypes important?
- They dictate what operations can be performed and prevent bad behaviours



Key concepts: types, functions, loops

- R primarily uses “atomic vectors” – these are homogenous vectors

Vector	Stores	Syntax
Logical	Boolean	True T
Integer	Whole numbers	1L, 2L
Double	Decimals	1, 1.2
Character	Text	“Hello!”
Complex	Real + imaginary	1 + 1i
Raw	Raw bytes	As.raw(1) = 01

You will most likely
only use these

A great explanation on
complex numbers

<https://betterexplained.com/articles/a-visual-intuitive-guide-to-imaginary-numbers/>

Only if you need binary hex



Key concepts: types, functions, loops

Term	R structure
Scalar	1
Vector	<code>c(1, 2)</code> <pre>[1] 1 2</pre>
Matrix	<code>matrix(1:4, nrow = 2)</code> <pre> [,1] [,2] [1,] 1 3 [2,] 2 4</pre>
3D+ array	<code>data_sequence <- 1:24</code> <code>3d_array <- array(data_sequence, dim = c(3, 4, 2))</code> <pre> [,1] [,2] [,3] [,4] [1,] 1 4 7 10 [2,] 2 5 8 11 [3,] 3 6 9 12 [,1] [,2] [,3] [,4] [1,] 13 16 19 22 [2,] 14 17 20 23 [3,] 15 18 21 24</pre>

Key concepts: types, functions, loops

- Functions:
- Self-contained blocks of code that perform a single “task”
- Good for performing the same task multiple times
- **"Don't repeat yourself"**
- Say you want to perform a MWU and graph a box plot but want the figures to *look* slightly differently – bake this into the function

```
basic_boxplot <- function(data, x_var, y_var) {...}
```

```
better_boxplot <- function(data, x_var, y_var, title = "this is my title" ...) {...}
```



Key concepts: types, functions, loops

- Loops:
- Repetitive function that is repeated until a condition is met
 - This could be until n is exhausted or something inbuilt into the loop which *breaks* it
- Good for iterative tasks

For

```
running_sum <- 0
for (i in 1:5) {
  running_sum <- running_sum + i
  print(running_sum)
}
```

[1]	1
[1]	3
[1]	6
[1]	10
[1]	15

While

```
count <- 1
while (count <= 3) {
  print(count)
  count <- count + 1
}
```

[1]	1
[1]	2
[1]	3

Repeat

```
i <- 1
repeat {
  print(i)
  i <- i + 1
  if (i > 3) {
    break
  }
}
```

[1]	1
[1]	2
[1]	3



Key concepts: types, functions, loops

- R primarily uses “atomic vectors” – these are homogenous vectors
- R will try to coerce a vector:
 - Logical -> int -> double -> complex -> character
- For example, python does not enforce this.
- This is due to design philosophy of the language
- You can however make a “list” which can be composed of any R object

```
> c(TRUE, 2L)
[1] 1 2
> c(1L, 2.5)
[1] 1.0 2.5
```

```
R_list = list(
  1,
  "hi",
  list(key = 1),
  list("list_2")
)

[[1]]
[1] 1

[[2]]
[1] "hi"

[[3]]
[[3]]$key
[1] 1

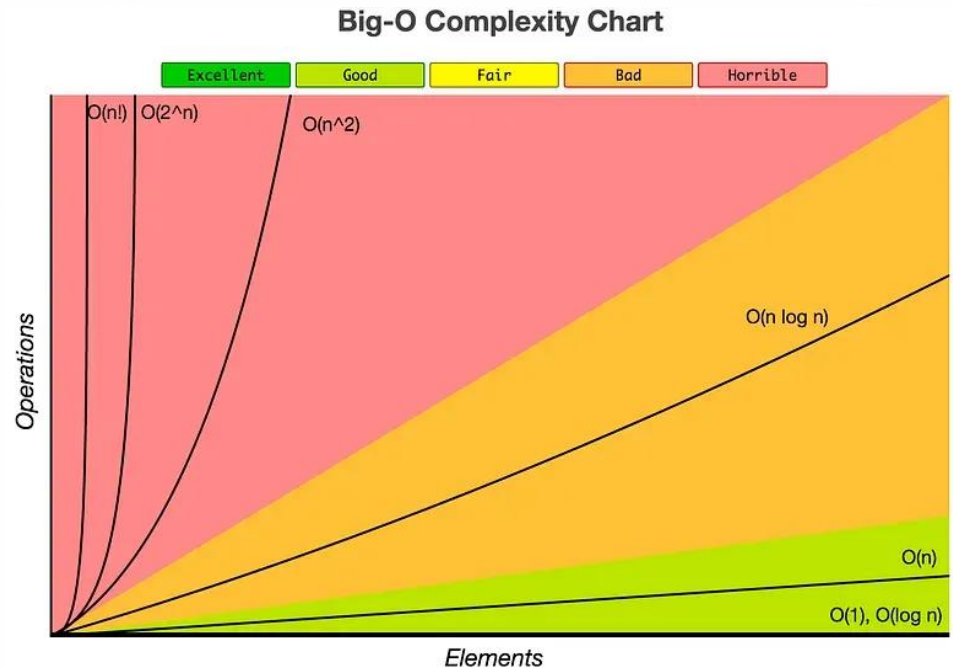
[[4]]
[[4]][[1]]
[1] "list_2"
```

```
python_list = [
  1, # int
  "hi", # character
  {"key" : 1}, # dict
  ["list_2"]
]

[1, 'hi', {'key': 1}, ['list_2']]
```



Key concepts: reducing footprint



Common Data Structure Operations

Data Structure	Time Complexity								Space Complex
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Singly-Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Doubly-Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Skip List	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n \log(n))$
Hash Table	N/A	$O(1)$	$O(1)$	$O(1)$	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Binary Search Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Cartesian Tree	N/A	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	N/A	$O(n)$	$O(n)$	$O(n)$	$O(n)$
B-Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
Red-Black Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
Splay Tree	N/A	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	N/A	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
AVL Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$
KD Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$

From: <https://medium.com/@gokulkrishnanpv/understanding-time-and-space-complexity-in-dsa-f8bf1b585129>



Australian and New Zealand
SOCIETY FOR IMMUNOLOGY INC.

The premier scientific society for Immunology research in Australia and New Zealand

Lets move to R

- Load R!
- Download data from:
<https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE168944>

